ĐẠI HỌC QUỐC GIA TPHCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

CSC14118 - Nhập môn Big Data
Báo cáo thực hành môn học

Chủ đề: Lab 01: Hadoop Cluster và WordCount

Sinh viên thực hiện:
Trần Hữu Kim Thành (23120166)

Giảng viên hướng dẫn:
Th.S Trần Huy Bân

Ngày 2 tháng 4 năm 2026

Mục lục

1 Giới thiệu **1**

1.1 Môi trường thực hành 1

1.2 Các phiên bản phần mềm sử dụng 1

2 Cài đặt Hadoop chế độ giả phân tán (Pseudo-Distributed Mode) **2**

2.1 Bước 1: Cài đặt môi trường Java, SSH và pdsh 2

2.2 Bước 2: Thiết lập SSH không cần mật khẩu 3

2.3 Bước 3: Tải và giải nén Apache Hadoop 3.3.6 4

2.4 Bước 4: Kiểm tra phiên bản và thiết lập biến môi trường 5

2.5 Bước 5: Cấu hình các file XML của Hadoop 6

2.5.1 File `hadoop-env.sh` 6

2.5.2 File `core-site.xml` 7

2.5.3 File `hdfs-site.xml` 7

2.5.4 File `mapred-site.xml` và `yarn-site.xml` 8

2.6 Bước 6: Khởi tạo HDFS (Format NameNode) 9

2.7 Bước 7: Xử lý sự cố `pdsh` và khởi động cụm 10

2.7.1 Lỗi kết nối `pdsh` (Permission denied) 10

2.7.2 Tra cứu và tìm phương án khắc phục 10

2.7.3 Phương án khắc phục 11

2.7.4 Khởi động lại tiến trình của Hadoop 11

2.8 Bước 8: Khảo sát hệ thống qua Web UI 12

2.9 Bước 9: Thao tác tệp trên HDFS và kiểm tra phân quyền 13

2.10 Bước 10: Vận hành cơ chế sinh tệp kiểm tra (Verification) 14

2.10.1 Chuẩn bị file JAR 14

2.10.2 Lần chạy 1 — Lỗi phân quyền (Permission denied) 15

2.10.3 Lần chạy 2 — Lỗi quyền sở hữu (Owner mismatch) 15

2.10.4 Lần chạy 3 — Thành công, sinh file Verification 16

2.10.5 Kiểm tra file kết quả 17

3 Bài toán đếm từ (Word Count) với bộ lọc ký tự đầu bằng Scala **18**

3.1 Bước 1: Chuẩn bị dữ liệu đầu vào trên HDFS 18

3.2 Bước 2: Kiểm tra phiên bản Scala 18

3.3 Bước 3: Thiết kế chương trình MapReduce bằng Scala 19
3.3.1 Lớp TokenizerMapper 19
3.3.2 Lớp IntSumReducer 20
3.3.3 Object FilterWordCount (Driver) 21
3.4 Bước 4: Biên dịch mã nguồn Scala và đóng gói JAR 21
3.5 Bước 5: Chạy MapReduce Job và xử lý sự cố ClassPath 22
3.5.1 Lần chạy 1 — Lỗi ClassNotFoundException (Scala Runtime) 22
3.5.2 Tra cứu giải pháp 23
3.5.3 Phương án khắc phục 24
3.6 Bước 6: Theo dõi tiến trình và kết quả Job 25
3.7 Bước 7: Đọc và lưu kết quả 25

4 Kết luận **26**
4.1 Tóm tắt kết quả 26
4.2 Kinh nghiệm rút ra 27

A Phụ lục – Mã nguồn đầy đủ **28**
A.1 FilterWordCount.scala 28

Danh sách hình vẽ

1 Quá trình cài đặt đồng thời OpenJDK 11, SSH và pdsh qua `apt` 2
2 Kết quả tạo cặp khóa RSA 3072-bit và fingerprint tương ứng 3
3 SSH vào localhost thành công — hệ thống đã thêm host vào danh sách known hosts và đăng nhập trực tiếp không cần mật khẩu 4
4 Tải thành công Hadoop 3.3.6 (696 MB) từ máy chủ Apache với tốc độ 7.25 MB/s 5
5 Giải nén và đổi tên thư mục Hadoop về `~/hadoop` 5
6 Xác nhận Java 11 và Hadoop 3.3.6 đã được nhận diện chính xác sau khi cấu hình 6
7 Chỉnh sửa `hadoop-env.sh` với `nano` để khai báo `JAVA_HOME` 6
8 Chỉnh sửa `core-site.xml` — cấu hình NameNode tại `hdfs://localhost:9000` 7
9 Chỉnh sửa `hdfs-site.xml` — đặt hệ số nhân bản bằng 1 8
10 Chỉnh sửa `~/.bashrc` để khai báo đầy đủ biến môi trường Hadoop 8
11 Di chuyển thư mục Hadoop, áp dụng `source ~/.bashrc` và xác nhận cấu hình . 9
12 Tổng quan quá trình chỉnh sửa tuần tự các file cấu hình với `nano` trước khi format 9
13 Thông báo *"Storage directory has been successfully formatted"* — HDFS sẵn sàng hoạt động 9
14 Lỗi `rcmd: socket: Permission denied` khi chạy `start-dfs.sh` với `pdsh` mặc định dùng `rsh` 10
15 Giải pháp từ Stack Overflow: đặt `HADOOP_CLASSPATH` hoặc `PDSH_RCMD_TYPE=ssh` để khắc phục lỗi `pdsh` 10
16 Thêm `export PDSH_RCMD_TYPE=ssh` vào `~/.bashrc` để sửa lỗi `pdsh` 11
17 Lệnh `start-dfs.sh` sau khi fix — NameNode, DataNode và SecondaryNameNode khởi động thành công 12
18 Lệnh `jps` trả về đủ 5 thành phần lõi: DataNode, NameNode, SecondaryNameNode, ResourceManager và NodeManager 12
19 Trang tổng quan HDFS tại `localhost:9870` — hiển thị trạng thái Active, thông tin dung lượng lưu trữ 13
20 Tab Datanodes xác nhận 1 DataNode đang ở trạng thái *In Service* — cụm giả phân tán đang hoạt động bình thường 13
21 Thực hiện thành công chuỗi lệnh tạo thư mục, upload file và cấp quyền trên HDFS 14
22 Thư mục home của WSL từ Windows Explorer — có đủ file `hadoop-test.jar` và `test_file.txt` 15

23 Lần chạy 1: Chương trình yêu cầu quyền 744 nhưng chưa được đặt đệ quy . . . 15
24 Lần chạy 2: Quyền 744 đã đúng nhưng chủ sở hữu file HDFS vẫn chưa khớp với `khtn_23120166` . . . 16
25 Lần chạy 3: Chương trình trả về *"Your student ID: 23120166"* — cảnh báo MAC bình thường trong WSL, cơ chế dự phòng hoạt động và sinh file `23120166_verification.txt` thành công . . . 17
26 File `23120166_verification.txt` chứa địa chỉ MAC `00-15-5D-35-19-75` — minh chứng hoàn thành bài thực hành trên máy cụ thể . . . 17
27 File `words.txt` (5,329,535 bytes) đã được upload thành công vào `/hcmus/23120166/input/` trên HDFS . . . 18
28 Scala 2.11.12 đã được cài đặt sẵn trên hệ thống — phiên bản tương thích với môi trường Hadoop . . . 19
29 VSCode hiển thị kết quả biên dịch — 7 file `.class` được tạo ra tương ứng với các lớp và anonymous functions . . . 22
30 Lệnh `jar -cvf` đóng gói thành công tất cả các file `.class` vào `WordCount.jar` . 22
31 MapReduce Job thất bại với `NoClassDefFoundError: scala/collection/Seq` — thiếu Scala Runtime Library trong Hadoop classpath . . . 23
32 Giải pháp từ Stack Overflow: thêm `scala-library.jar` vào `HADOOP_CLASSPATH` để Hadoop tìm thấy Scala Runtime . . . 24
33 Sau khi export `HADOOP_CLASSPATH` với `scala-library.jar`, Job được submit thành công và bắt đầu thực thi . . . 24
34 Job hoàn thành với `map 100% reduce 100%` và thông báo *"Job completed successfully"* — tất cả 36 counters được ghi nhận . . . 25
35 Kết quả đếm từ: `f=15870, i=15220, t=25223, h=18662, c=38934, m=25194, u=23790, s=50571` — xuất theo đúng thứ tự FIT HCMUS . . . 25
36 Windows Explorer xác nhận file `results.txt` đã được lưu về máy local, cùng với kết quả đếm từ hiển thị bên phải . . . 26

1 Giới thiệu

Báo cáo này trình bày quá trình thực hành Lab 1 của môn học **Nhập môn Big Data (CSC14118)**, bao gồm hai nội dung chính:

1. **Cài đặt cụm Hadoop giả phân tán (pseudo-distributed mode)** trên môi trường WSL/Ubuntu, bao gồm cấu hình Java, SSH, HDFS, và YARN.
2. **Thực hiện bài toán đếm từ (Word Count)** sử dụng ngôn ngữ Scala với framework MapReduce trên nền tảng Hadoop.

1.1 Môi trường thực hành

Toàn bộ quá trình cài đặt và thực hành được thực hiện trên **Windows Subsystem for Linux (WSL)** với bản phân phối **Ubuntu 22.04 LTS**. Lý do chọn môi trường này:

- WSL cho phép chạy môi trường Linux đầy đủ trên Windows mà không cần máy ảo, giúp tiết kiệm tài nguyên phần cứng.
- Ubuntu 22.04 là bản LTS ổn định, được hỗ trợ dài hạn và tương thích tốt với Hadoop.
- Thống nhất với các thành viên trong nhóm để dễ dàng phối hợp và debug khi gặp lỗi.

1.2 Các phiên bản phần mềm sử dụng

Phần mềm	**Phiên bản**	**Lý do chọn**
Java (OpenJDK)	11	Được nhóm thống nhất; tương thích Hadoop 3.x
Hadoop	3.3.6	Phổ biến, ổn định, nhiều tài liệu hỗ trợ
Scala	2.11.12	Tương thích Hadoop 3.x; dùng trong MapReduce

Bảng 1: Các phiên bản phần mềm sử dụng trong Lab

Lý do chọn Java 11: Nhóm đã thống nhất sử dụng Java 11 vì đây là phiên bản LTS được Hadoop 3.3.x hỗ trợ chính thức, và tất cả thành viên đều có sẵn môi trường này từ các môn học trước.

Lý do chọn Hadoop 3.3.6: Đây là bản phát hành ổn định nhất tại thời điểm thực hành, có nhiều bản vá lỗi so với các phiên bản trước, và được cộng đồng sử dụng rộng rãi.

2 Cài đặt Hadoop chế độ giả phân tán (Pseudo-Distributed Mode)

Trong phần này, báo cáo trình bày chi tiết từng bước cấu hình Hadoop theo chế độ **Pseudo-Distributed Mode**. Đây là chế độ triển khai mô phỏng một cụm phân tán (cluster) trên một máy tính vật lý duy nhất. Trong đó, mỗi tiến trình (daemon) của Hadoop như NameNode, DataNode, ResourceManager và NodeManager sẽ được khởi chạy độc lập dưới dạng một tiến trình Java riêng biệt, giúp người phát triển dễ dàng kiểm thử các ứng dụng Big Data trước khi triển khai thực tế.

2.1 Bước 1: Cài đặt môi trường Java, SSH và pdsh

Mục đích: Hadoop được viết bằng ngôn ngữ Java, do đó môi trường thực thi Java (Java Development Kit) là thành phần bắt buộc. Bên cạnh đó, Hadoop sử dụng giao thức SSH để điều phối các tiến trình daemon giữa các node, và công cụ `pdsh` (Parallel Distributed Shell) để chạy lệnh đồng thời trên nhiều node. Trong môi trường giả phân tán, SSH cũng được dùng để kết nối vào `localhost`.

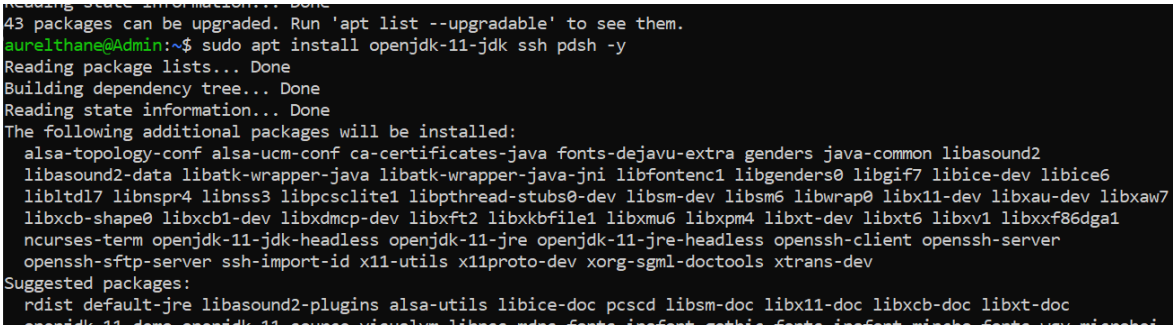
Các lệnh thực hiện:

1. `sudo apt update`: Cập nhật danh sách các gói phần mềm mới nhất từ kho lưu trữ của Ubuntu.

2. `sudo apt install openjdk-11-jdk ssh pdsh -y`: Cài đặt đồng thời Java 11 JDK, OpenSSH server/client và công cụ pdsh, tham số `-y` để tự động đồng ý.

```
sudo apt update
sudo apt install openjdk-11-jdk ssh pdsh -y
```

Listing 1: Cài đặt Java 11, SSH và pdsh trên Ubuntu



Hình 1: Quá trình cài đặt đồng thời OpenJDK 11, SSH và pdsh qua `apt`

2.2 Bước 2: Thiết lập SSH không cần mật khẩu

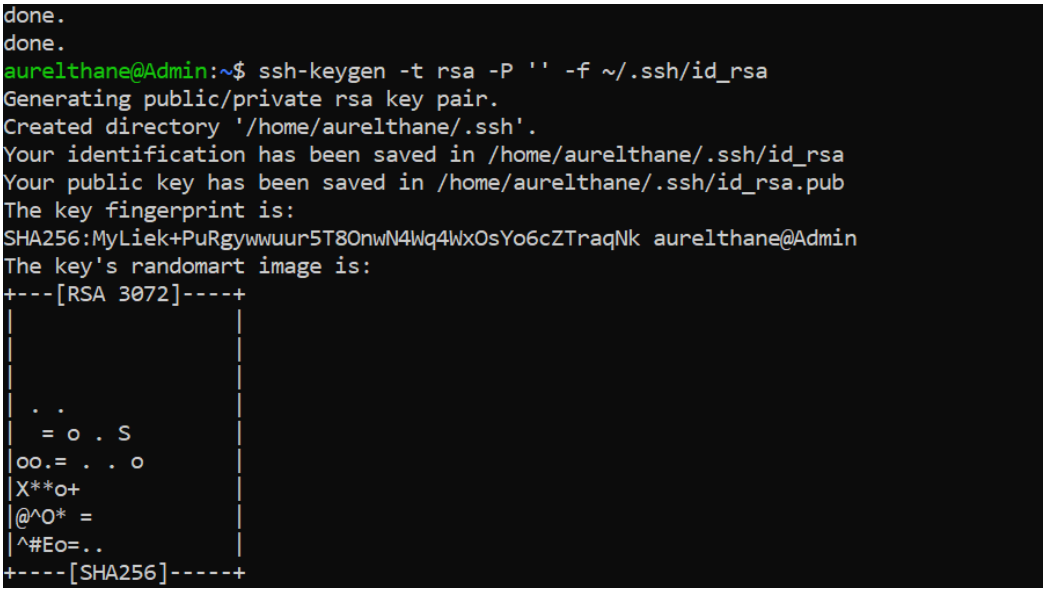
Mục đích: Hadoop cần sử dụng chuẩn kết nối SSH để quản lý các node (thậm chí là trên cùng một máy localhost). Việc thiết lập SSH không cần mật khẩu giúp các lệnh khởi động tự động (như `start-dfs.sh`) có thể làm việc trơn tru mà không bị gián đoạn bởi yêu cầu nhập mật khẩu.

Các lệnh thực hiện:

1. `ssh-keygen`: Tạo cặp khóa RSA (`-t rsa`), không đặt mật khẩu (`-P ''`).
2. `cat ... » authorized_keys`: Đưa public key vào danh sách khóa được phép truy cập tự động.
3. `chmod 0600`: Đặt quyền file chỉ đọc/ghi bởi chủ sở hữu (yêu cầu bảo mật SSH).

```
ssh-keygen -t rsa -P '' -f ~/.ssh/id_rsa
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 0600 ~/.ssh/authorized_keys
```

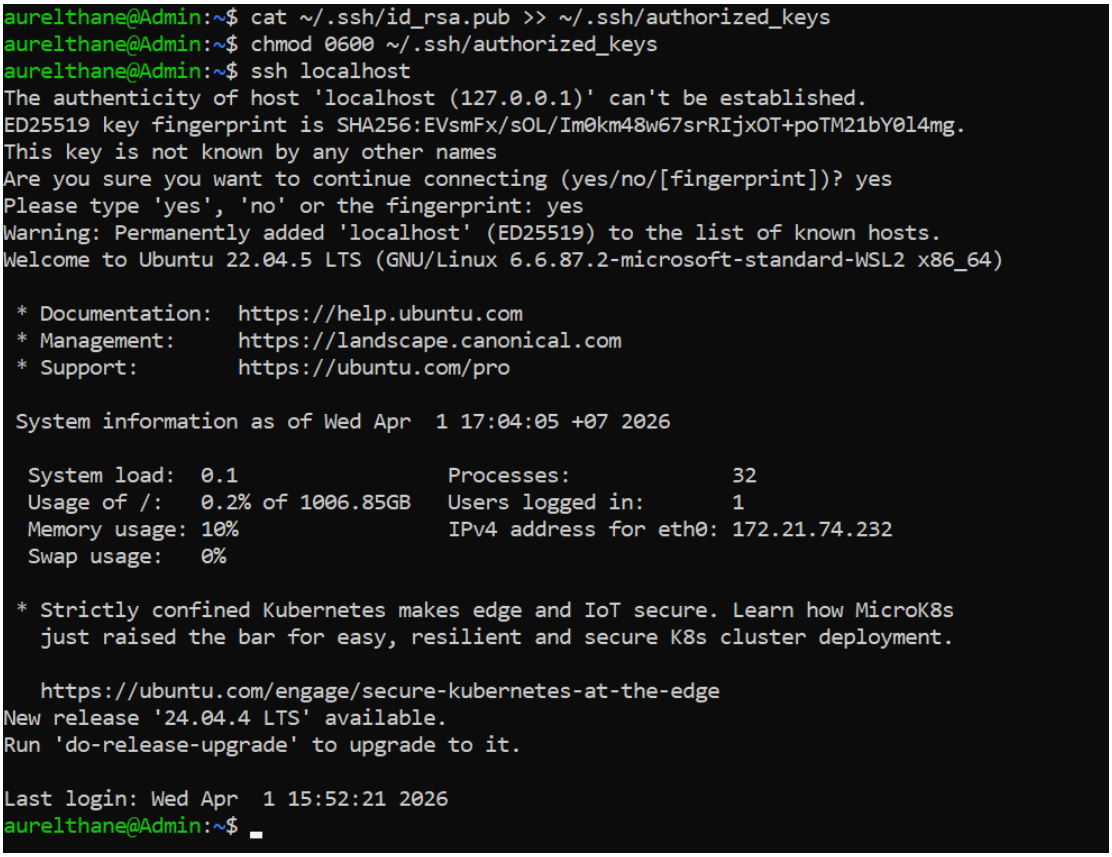
Listing 2: Tạo SSH key và cấu hình đăng nhập tự động vào localhost



```
done.
done.
aurelthane@Admin:~$ ssh-keygen -t rsa -P '' -f ~/.ssh/id_rsa
Generating public/private rsa key pair.
Created directory '/home/aurelthane/.ssh'.
Your identification has been saved in /home/aurelthane/.ssh/id_rsa
Your public key has been saved in /home/aurelthane/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:MyLiek+PuRgywwuur5T8OnwN4Wq4WxOsYo6cZTraqNk aurelthane@Admin
The key's randomart image is:
+---[RSA 3072]----+
|                 |
|                 |
|                 |
| . .             |
|  = o . S        |
|oo.= . . o       |
|X**o+            |
|@^O* =           |
|^#Eo=..          |
+----[SHA256]-----+
```

Hình 2: Kết quả tạo cặp khóa RSA 3072-bit và fingerprint tương ứng

Kiểm tra kết nối SSH: Thực hiện lệnh `ssh localhost` để xác nhận kết nối thành công không yêu cầu mật khẩu.



Hình 3: SSH vào localhost thành công — hệ thống đã thêm host vào danh sách known hosts và đăng nhập trực tiếp không cần mật khẩu

2.3 Bước 3: Tải và giải nén Apache Hadoop 3.3.6

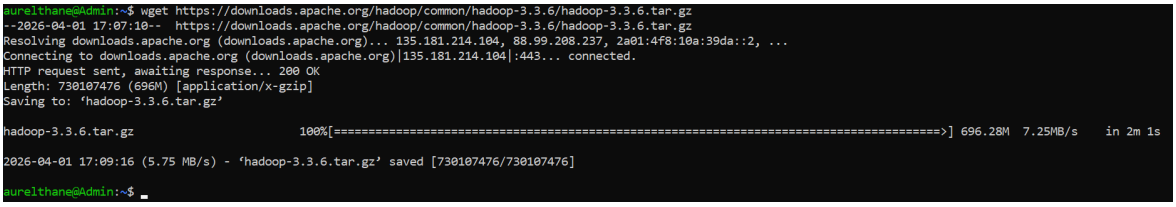
Mục đích: Lấy bộ mã nguồn đã biên dịch của Hadoop từ máy chủ chính thức của Apache và thiết lập thư mục làm việc chính cho Hadoop trên hệ thống (tại `~/hadoop`).

Các lệnh thực hiện:

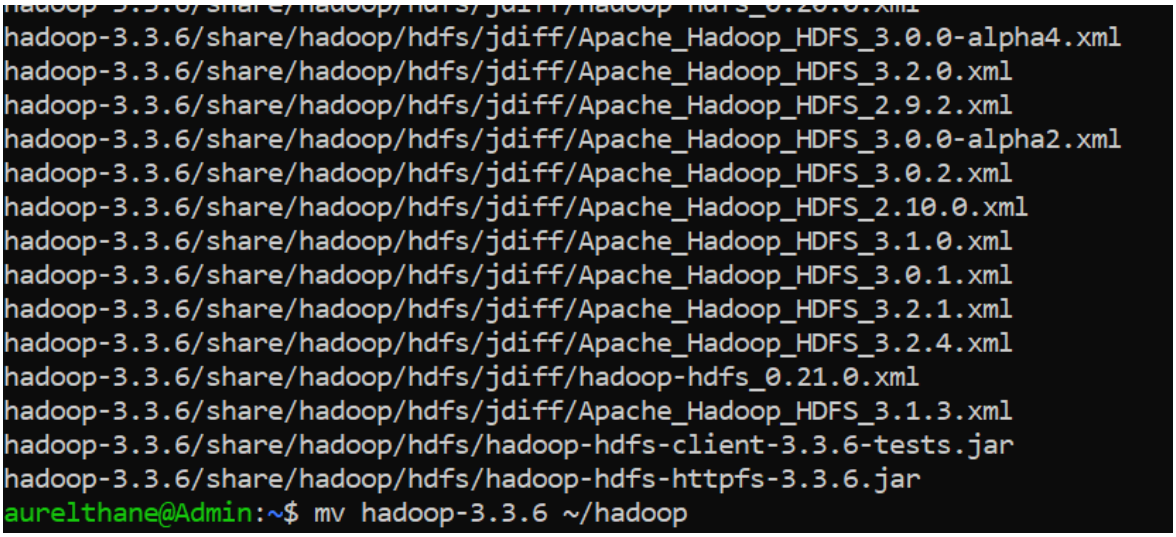
1. `wget`: Lệnh tải file `tar.gz` từ URL chỉ định của Apache.
2. `tar -xzf`: Giải nén file (`-x`: extract, `-z`: qua gzip, `-f`: chỉ định file).
3. `mv`: Đổi tên thư mục mặc định sang `~/hadoop` để thuận tiện cấu hình.

```
wget https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar
    .gz
tar -xzf hadoop-3.3.6.tar.gz
mv hadoop-3.3.6 ~/hadoop
```

Listing 3: Tải và giải nén Hadoop 3.3.6



Hình 4: Tải thành công Hadoop 3.3.6 (696 MB) từ máy chủ Apache với tốc độ 7.25 MB/s



Hình 5: Giải nén và đổi tên thư mục Hadoop về `~/hadoop`

2.4 Bước 4: Kiểm tra phiên bản và thiết lập biến môi trường

Mục đích: Biến môi trường giúp hệ điều hành biết vị trí cài đặt Java và Hadoop để có thể gọi các lệnh như `hdfs`, `hadoop`, `start-dfs.sh` từ bất kỳ thư mục nào.

Chi tiết cấu hình trong `~/.bashrc`:

- `JAVA_HOME`: Trỏ tới thư mục cài đặt gốc của OpenJDK 11.
- `HADOOP_HOME`: Trỏ tới thư mục vừa giải nén.
- `PATH`: Bổ sung các thư mục `bin` và `sbin` của Hadoop.
- `HADOOP_CONF_DIR`: Trỏ tới thư mục chứa các file cấu hình XML.

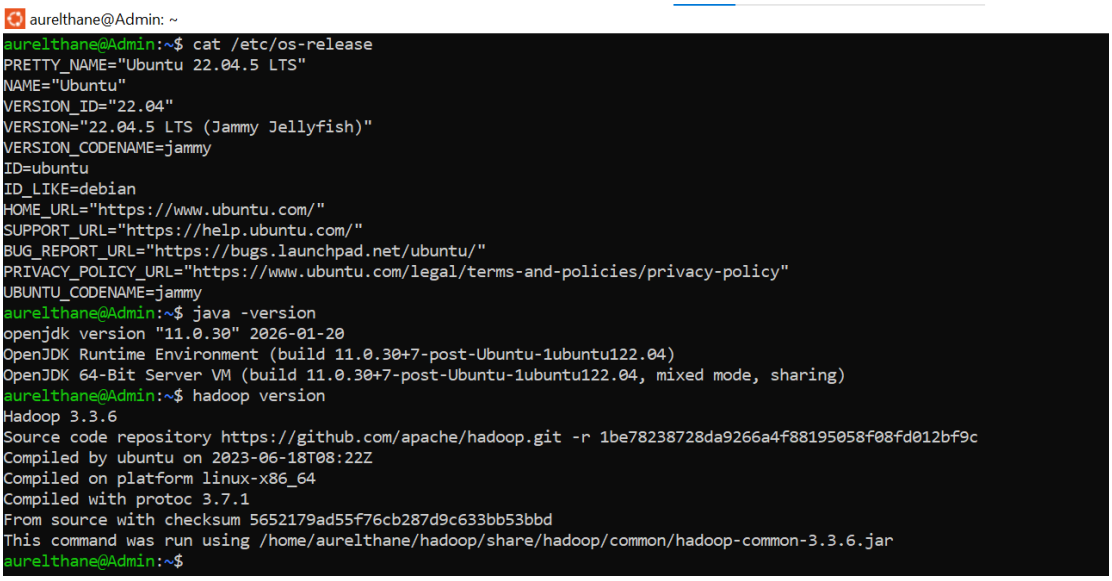
```
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
export HADOOP_HOME=~/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop
```

Listing 4: Cấu hình biến môi trường trong ~/.bashrc

Áp dụng và kiểm tra: Sau khi lưu file, dùng `source` để áp dụng ngay, kiểm tra bằng `java -version` và `hadoop version`.

```
source ~/.bashrc
java -version
hadoop version
```

Listing 5: Áp dụng cấu hình và kiểm tra phiên bản



Hình 6: Xác nhận Java 11 và Hadoop 3.3.6 đã được nhận diện chính xác sau khi cấu hình

2.5 Bước 5: Cấu hình các file XML của Hadoop

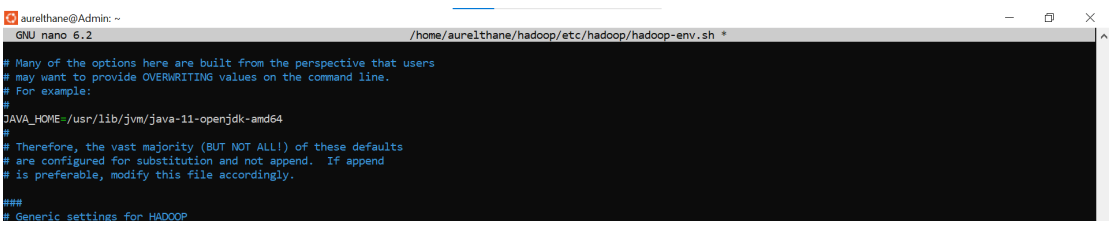
Hadoop được điều khiển bởi tập hợp các file XML nằm trong `$HADOOP_HOME/etc/hadoop/`.

2.5.1 File `hadoop-env.sh`

Mục đích: Khai báo cứng đường dẫn `JAVA_HOME` trong file cấu hình môi trường riêng của Hadoop, đảm bảo các tiến trình daemon sử dụng đúng JVM.

```
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
```

Listing 6: Khai báo JAVA_HOME trong hadoop-env.sh



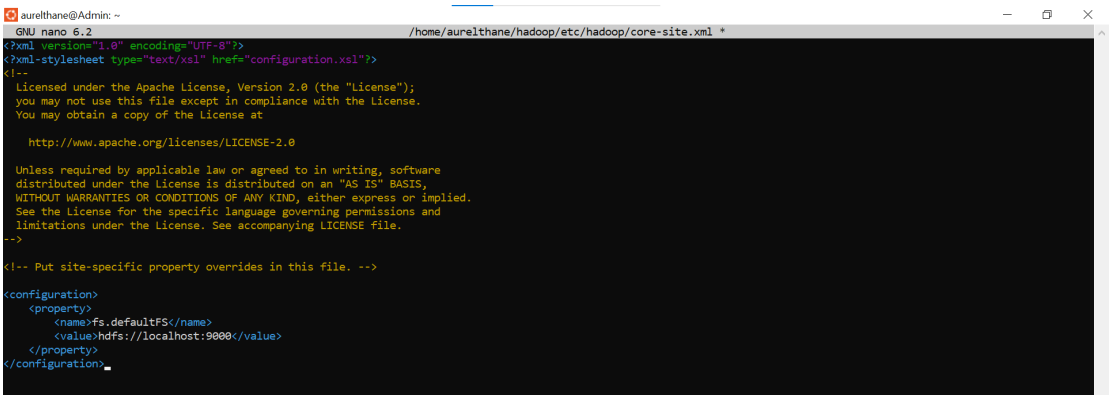
Hình 7: Chỉnh sửa `hadoop-env.sh` với `nano` để khai báo `JAVA_HOME`

2.5.2 File `core-site.xml`

Mục đích: Định nghĩa URI phân giải của filesystem mặc định (NameNode). Mọi truy vấn `hdfs://` sẽ được trỏ tới cổng 9000 trên localhost.

```xml
<configuration>
    <property>
        <name>fs.defaultFS</name>
        <value>hdfs://localhost:9000</value>
    </property>
</configuration>
```

Listing 7: Nội dung file core-site.xml



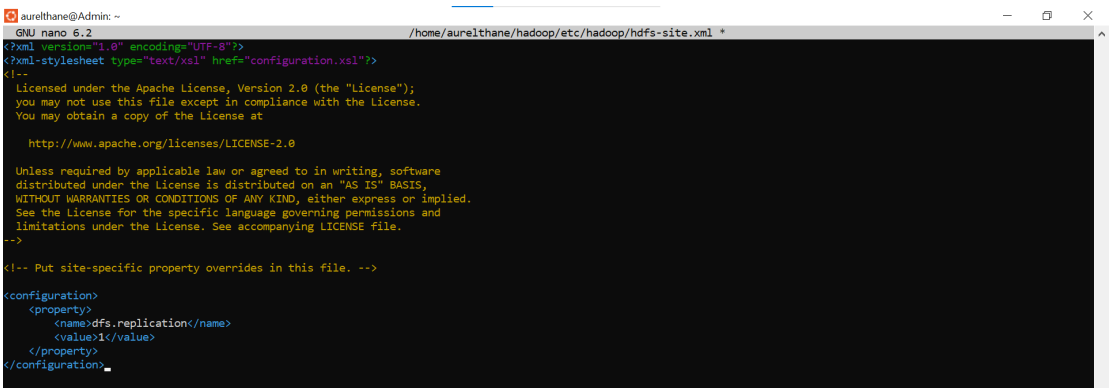
Hình 8: Chỉnh sửa `core-site.xml` — cấu hình NameNode tại `hdfs://localhost:9000`

2.5.3 File `hdfs-site.xml`

Mục đích: Quản lý cấu hình HDFS. Trong chế độ giả phân tán chỉ có 1 DataNode nên hệ số nhân bản (Replication Factor) buộc phải là 1.

```xml
<configuration>
    <property>
        <name>dfs.replication</name>
        <value>1</value>
    </property>
</configuration>
```

Listing 8: Nội dung file hdfs-site.xml



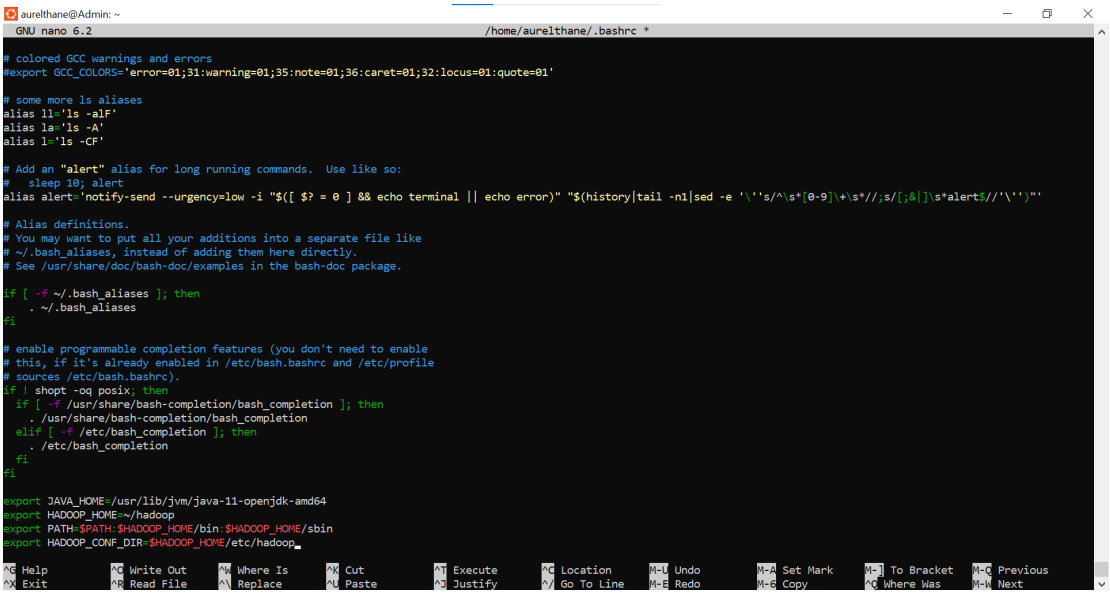
Hình 9: Chỉnh sửa `hdfs-site.xml` — đặt hệ số nhân bản bằng 1

2.5.4 File `mapred-site.xml` và `yarn-site.xml`

Mục đích: Cấu hình framework tính toán phân tán. Sử dụng YARN làm hệ thống quản lý tài nguyên và bộ điều phối tác vụ MapReduce.

```xml
<configuration>
    <property>
        <name>mapreduce.framework.name</name>
        <value>yarn</value>
    </property>
</configuration>
```

Listing 9: Tích hợp YARN trong mapred-site.xml



Hình 10: Chỉnh sửa `~/.bashrc` để khai báo đầy đủ biến môi trường Hadoop

```
hadoop-3.3.6/share/hadoop/hdfs/hadoop-hdfs-httpfs-3.3.6.jar
aurelthane@Admin:~$ mv hadoop-3.3.6 ~/hadoop
aurelthane@Admin:~$ nano ~/.bashrc
aurelthane@Admin:~$ source ~/.bashrc
aurelthane@Admin:~$
```

Hình 11: Di chuyển thư mục Hadoop, áp dụng `source ~/.bashrc` và xác nhận cấu hình

2.6 Bước 6: Khởi tạo HDFS (Format NameNode)

Mục đích: Trước khi HDFS có thể hoạt động lần đầu tiên, hệ thống cần tạo cấu trúc metadata gốc bao gồm file `fsimage` (ảnh chụp trạng thái namespace) và thư mục lưu trữ local của NameNode. Thao tác này chỉ thực hiện một lần duy nhất và không được lặp lại khi đã có dữ liệu.

```
hdfs namenode -format
```

Listing 10: Format HDFS lần đầu

```
aurelthane@Admin:~$ source ~/.bashrc
aurelthane@Admin:~$ nano ~/hadoop/etc/hadoop/hadoop-env.sh
aurelthane@Admin:~$ nano ~/hadoop/etc/hadoop/core-site.xml
aurelthane@Admin:~$ nano ~/hadoop/etc/hadoop/hdfs-site.xml
aurelthane@Admin:~$
```

Hình 12: Tổng quan quá trình chỉnh sửa tuần tự các file cấu hình với `nano` trước khi format

```
Select aurelthane@Admin: ~
2026-04-01 17:21:22,953 INFO namenode.FSDirectory: USER serial map: bits=24 maxEntries=16777215
2026-04-01 17:21:22,953 INFO namenode.FSDirectory: GROUP serial map: bits=24 maxEntries=16777215
2026-04-01 17:21:22,953 INFO namenode.FSDirectory: XATTR serial map: bits=24 maxEntries=16777215
2026-04-01 17:21:22,964 INFO util.GSet: Computing capacity for map INodeMap
2026-04-01 17:21:22,964 INFO util.GSet: VM type       = 64-bit
2026-04-01 17:21:22,964 INFO util.GSet: 1.0% max memory 924 MB = 9.2 MB
2026-04-01 17:21:22,964 INFO util.GSet: capacity      = 2^20 = 1048576 entries
2026-04-01 17:21:22,967 INFO namenode.FSDirectory: ACLs enabled? true
2026-04-01 17:21:22,967 INFO namenode.FSDirectory: POSIX ACL inheritance enabled? true
2026-04-01 17:21:22,968 INFO namenode.FSDirectory: XAttrs enabled? true
2026-04-01 17:21:22,968 INFO namenode.NameNode: Caching file names occurring more than 10 times
2026-04-01 17:21:22,972 INFO snapshot.SnapshotManager: Loaded config captureOpenFiles: false, skipCaptureAccessTimeOnlyChange: false, snapshotDiffAllowSnapRootDescendant: t
rue, maxSnapshotLimit: 65536
2026-04-01 17:21:22,973 INFO snapshot.SnapshotManager: SkipList is disabled
2026-04-01 17:21:22,978 INFO util.GSet: Computing capacity for map cachedBlocks
2026-04-01 17:21:22,978 INFO util.GSet: VM type       = 64-bit
2026-04-01 17:21:22,978 INFO util.GSet: 0.25% max memory 924 MB = 2.3 MB
2026-04-01 17:21:22,978 INFO util.GSet: capacity      = 2^18 = 262144 entries
2026-04-01 17:21:22,998 INFO metrics.TopMetrics: NNTop conf: dfs.namenode.top.window.num.buckets = 10
2026-04-01 17:21:22,998 INFO metrics.TopMetrics: NNTop conf: dfs.namenode.top.num.users = 10
2026-04-01 17:21:22,998 INFO metrics.TopMetrics: NNTop conf: dfs.namenode.top.windows.minutes = 1,5,25
2026-04-01 17:21:23,003 INFO namenode.FSNamesystem: Retry cache on namenode is enabled
2026-04-01 17:21:23,004 INFO namenode.FSNamesystem: Retry cache will use 0.03 of total heap and retry cache entry expiry time is 600000 millis
2026-04-01 17:21:23,007 INFO util.GSet: Computing capacity for map NameNodeRetryCache
2026-04-01 17:21:23,007 INFO util.GSet: VM type       = 64-bit
2026-04-01 17:21:23,008 INFO util.GSet: 0.029999999329447746% max memory 924 MB = 283.9 KB
2026-04-01 17:21:23,008 INFO util.GSet: capacity      = 2^15 = 32768 entries
2026-04-01 17:21:23,029 INFO namenode.FSImage: Allocated new BlockPoolId: BP-115270631-127.0.1.1-1775038883021
2026-04-01 17:21:23,060 INFO common.Storage: Storage directory /tmp/hadoop-aurelthane/dfs/name has been successfully formatted.
2026-04-01 17:21:23,089 INFO namenode.FSImageFormatProtobuf: Saving image file /tmp/hadoop-aurelthane/dfs/name/current/fsimage.ckpt_0000000000000000000 using no compression
2026-04-01 17:21:23,172 INFO namenode.FSImageFormatProtobuf: Image file /tmp/hadoop-aurelthane/dfs/name/current/fsimage.ckpt_0000000000000000000 of size 405 bytes saved in
0 seconds .
2026-04-01 17:21:23,191 INFO namenode.NNStorageRetentionManager: Going to retain 1 images with txid >= 0
2026-04-01 17:21:23,208 INFO namenode.FSNamesystem: Stopping services started for active state
2026-04-01 17:21:23,209 INFO namenode.FSNamesystem: Stopping services started for standby state
2026-04-01 17:21:23,214 INFO namenode.FSImage: FSImageSaver clean checkpoint: txid=0 when meet shutdown.
2026-04-01 17:21:23,215 INFO namenode.NameNode: SHUTDOWN_MSG:
/************************************************************
SHUTDOWN_MSG: Shutting down NameNode at Admin/127.0.1.1
************************************************************/
aurelthane@Admin:~$
```

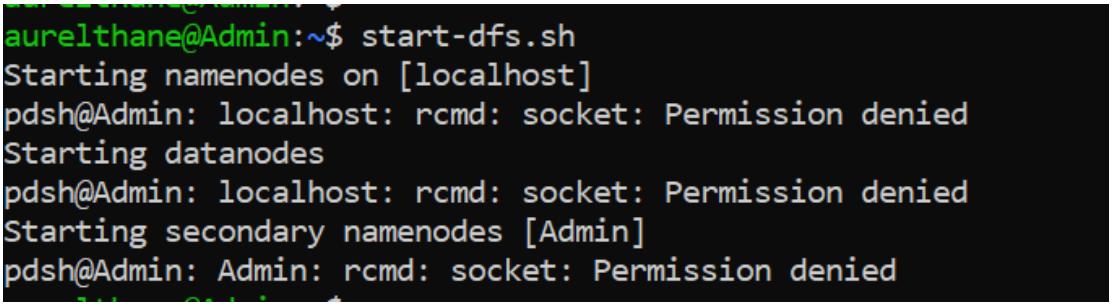
Hình 13: Thông báo *"Storage directory has been successfully formatted"* — HDFS sẵn sàng hoạt động

2.7 Bước 7: Xử lý sự cố pdsh và khởi động cụm

2.7.1 Lỗi kết nối pdsh (Permission denied)

Vấn đề: Khi thực thi `start-dfs.sh` để bật các thành phần lưu trữ, lệnh bị từ chối với thông báo `Permission denied`.

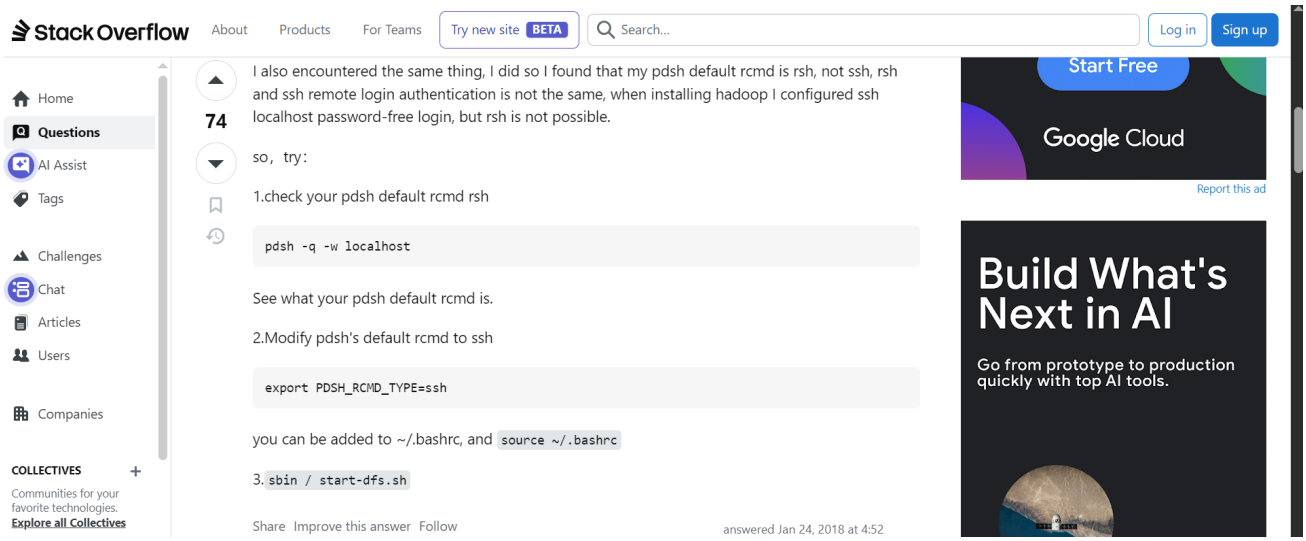
Nguyên nhân: Mặc định Hadoop sử dụng `pdsh` ưu tiên giao thức `rsh` (Remote Shell) thay vì `ssh`. Giao thức `rsh` không hỗ trợ xác thực bằng public key RSA đã thiết lập ở Bước 2, dẫn đến từ chối kết nối.



Hình 14: Lỗi `rcmd: socket: Permission denied` khi chạy `start-dfs.sh` với `pdsh` mặc định dùng `rsh`

2.7.2 Tra cứu và tìm phương án khắc phục

Tham khảo giải pháp từ cộng đồng Stack Overflow, phương án được đánh giá cao nhất là ép buộc `pdsh` sử dụng `ssh` thay vì `rsh` thông qua biến môi trường `PDSH_RCMD_TYPE`.



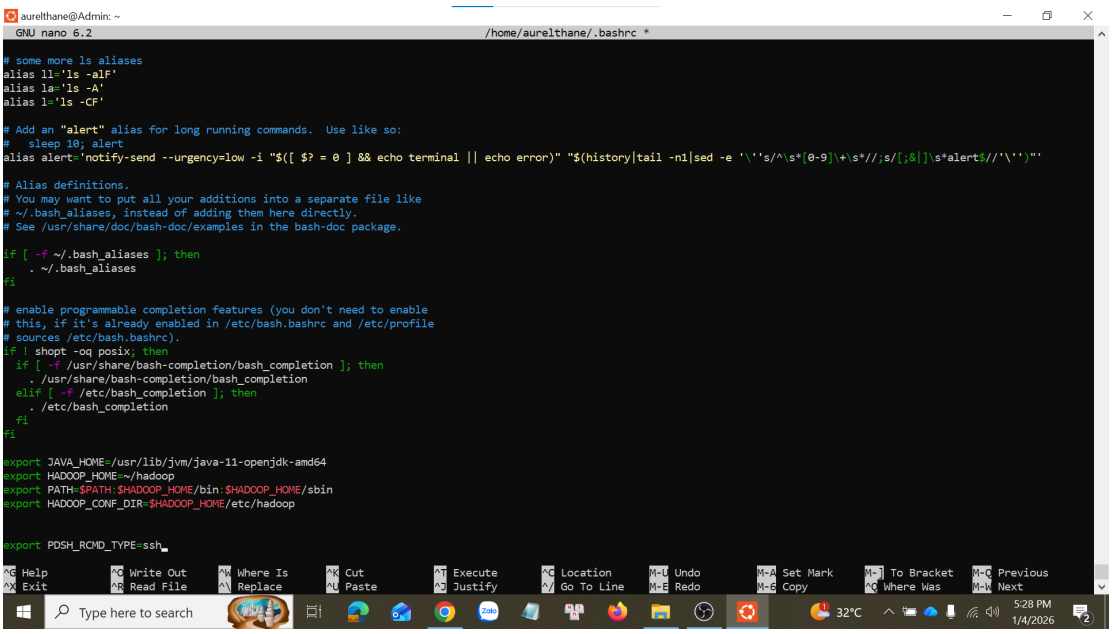
Hình 15: Giải pháp từ Stack Overflow: đặt `HADOOP_CLASSPATH` hoặc `PDSH_RCMD_TYPE=ssh` để khắc phục lỗi `pdsh`

2.7.3 Phương án khắc phục

Giải quyết: Thêm biến `PDSH_RCMD_TYPE=ssh` vào `~/.bashrc` để đảm bảo hiệu lực vĩnh viễn giữa các phiên làm việc.

```
echo 'export PDSH_RCMD_TYPE=ssh' >> ~/.bashrc
source ~/.bashrc
```

Listing 11: Khắc phục lỗi pdsh bằng cách chỉ định giao thức SSH



Hình 16: Thêm `export PDSH_RCMD_TYPE=ssh` vào `~/.bashrc` để sửa lỗi `pdsh`

2.7.4 Khởi động lại tiến trình của Hadoop

Sau khi áp dụng fix, lệnh `start-dfs.sh` khởi động thành công NameNode, DataNode và SecondaryNameNode. Tiếp theo khởi động YARN bằng `start-yarn.sh` và dùng `jps` để xác nhận tất cả daemon đang chạy.

```
start-dfs.sh
start-yarn.sh
jps
```

Listing 12: Khởi chạy DFS và YARN, kiểm tra tiến trình

```
aurelthane@Admin:~$ nano ~/.bashrc
aurelthane@Admin:~$ source ~/.bashrc
aurelthane@Admin:~$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [Admin]
```

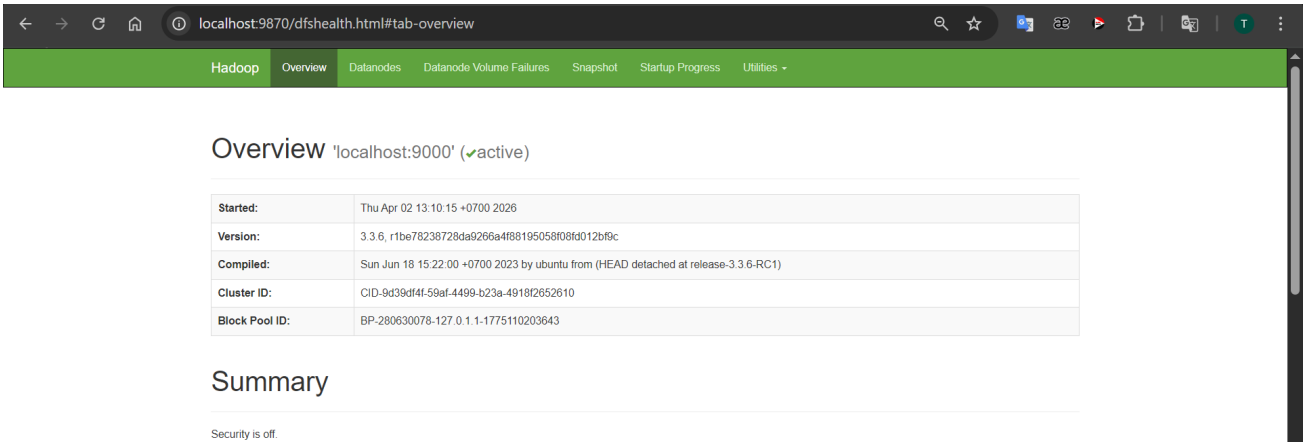
Hình 17: Lệnh `start-dfs.sh` sau khi fix — NameNode, DataNode và SecondaryNameNode khởi động thành công

```
aurelthane@Admin:~$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [Admin]
aurelthane@Admin:~$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
aurelthane@Admin:~$ jps
5457 NameNode
6036 ResourceManager
5828 SecondaryNameNode
5621 DataNode
6186 NodeManager
6523 Jps
aurelthane@Admin:~$
```

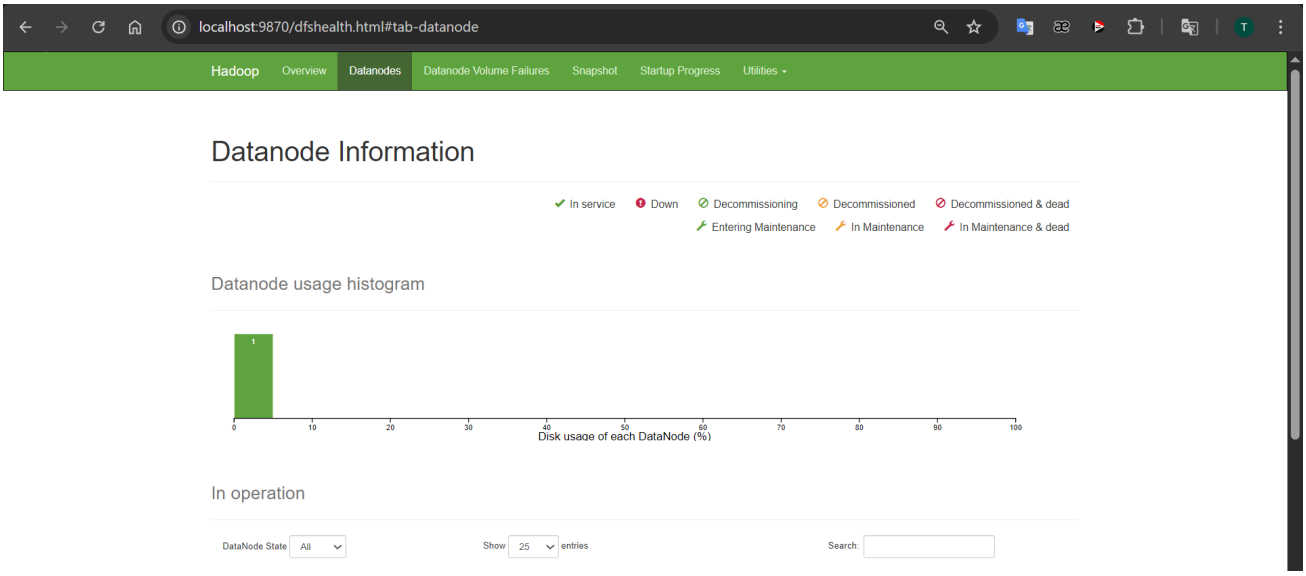
Hình 18: Lệnh `jps` trả về đủ 5 thành phần lõi: DataNode, NameNode, SecondaryNameNode, ResourceManager và NodeManager

2.8 Bước 8: Khảo sát hệ thống qua Web UI

Mục đích: HDFS cung cấp giao diện Web UI tại cổng **9870** để theo dõi trạng thái cụm theo thời gian thực. Đây là công cụ quan trọng để kiểm tra sức khỏe hệ thống, dung lượng lưu trữ, và số lượng DataNode đang hoạt động.



Hình 19: Trang tổng quan HDFS tại `localhost:9870` — hiển thị trạng thái Active, thông tin dung lượng lưu trữ



Hình 20: Tab Datanodes xác nhận 1 DataNode đang ở trạng thái *In Service* — cụm giả phân tán đang hoạt động bình thường

2.9 Bước 9: Thao tác tệp trên HDFS và kiểm tra phân quyền

Mục đích: Xây dựng hệ thống phân cấp thư mục trên HDFS, tạo người dùng và file thử nghiệm, đồng thời kiểm tra cơ chế quản lý quyền truy cập phân tán của Hadoop.

Các lệnh và ý nghĩa:

1. `hdfs dfs -mkdir`: Tạo cây thư mục logic trên HDFS cho mã số sinh viên.
2. `sudo useradd -m`: Tạo user Unix `khtn_23120166` với thư mục home.
3. `echo ... >` và `-put`: Tạo file local rồi đẩy lên HDFS bằng `-put`.

4. `-chmod 744`: Cấp quyền đọc/ghi cho chủ sở hữu, quyền đọc cho nhóm và người khác.

5. `-chown`: Bàn giao quyền sở hữu HDFS cho user `khtn_23120166`.

```
hdfs dfs -mkdir /hcmus
sudo useradd -m khtn_23120166
hdfs dfs -mkdir /hcmus/23120166
echo "Hello Big Data from 23120166" > test_file.txt
hdfs dfs -put test_file.txt /hcmus/23120166/
hdfs dfs -chmod 744 /hcmus/23120166
hdfs dfs -chown -R khtn_23120166 /hcmus/23120166
hdfs dfs -ls /hcmus
```

Listing 13: Tạo thư mục và thao tác file trên HDFS

```
aurelthane@Admin:~$ hdfs dfs -mkdir /hcmus
aurelthane@Admin:~$ sudo useradd -m khtn_23120166
[sudo] password for aurelthane:
aurelthane@Admin:~$ hdfs dfs -mkdir /hcmus/23120166
aurelthane@Admin:~$ pwd
/home/aurelthane
aurelthane@Admin:~$ echo "Hello Big Data from 23120166" > test_file.txt
aurelthane@Admin:~$ hdfs dfs -put test_file.txt /hcmus/23120166/
aurelthane@Admin:~$ hdfs dfs -chmod 744 /hcmus/23120166
aurelthane@Admin:~$ hdfs dfs -chown khtn_23120166 /hcmus/23120166
aurelthane@Admin:~$ hdfs dfs -ls /hcmus
Found 1 items
drwxr--r--   - khtn_23120166 supergroup          0 2026-04-01 17:37 /hcmus/23120166
```

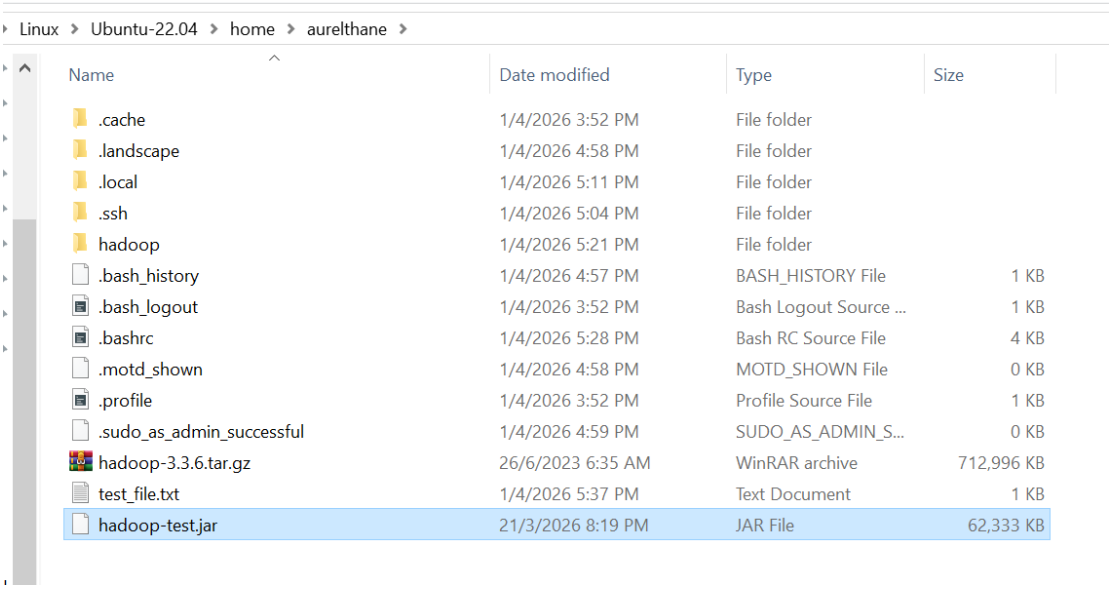
Hình 21: Thực hiện thành công chuỗi lệnh tạo thư mục, upload file và cấp quyền trên HDFS

2.10 Bước 10: Vận hành cơ chế sinh tệp kiểm tra (Verification)

Mục đích: Chạy chương trình Java kiểm tra (`hadoop-test.jar`) được cung cấp sẵn để xác minh Hadoop hoạt động đúng và sinh ra file định danh chứa địa chỉ MAC của máy.

2.10.1 Chuẩn bị file JAR

File `hadoop-test.jar` được đặt sẵn trong thư mục home của WSL. File này đọc dữ liệu từ đường dẫn HDFS được chỉ định, kiểm tra thông tin mạng và sinh ra file `23120166_verification.txt` chứa địa chỉ MAC.



Hình 22: Thư mục home của WSL từ Windows Explorer — có đủ file `hadoop-test.jar` và `test_file.txt`

2.10.2 Lần chạy 1 — Lỗi phân quyền (Permission denied)

```
java -jar hadoop-test.jar 9000 /hcmus/23120166
```

Listing 14: Chạy thử jar lần 1

Nguyên nhân: Lỗi `Ensure the permission is set to 744` — quyền trên thư mục HDFS chưa được đặt đệ quy.

```
aurelthane@Admin:~$ java -jar hadoop-test.jar 9000 /hcmus/23120166
Trying to read /hcmus/23120166
log4j:WARN No appenders could be found for logger (org.apache.hadoop.util.Shell).
log4j:WARN Please initialize the log4j system properly.
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.html#noconfig for more info.
Found hdfs://localhost:9000/hcmus/23120166/test_file.txt
Ensure the permission is set to 744
```

Hình 23: Lần chạy 1: Chương trình yêu cầu quyền 744 nhưng chưa được đặt đệ quy

2.10.3 Lần chạy 2 — Lỗi quyền sở hữu (Owner mismatch)

Sau khi đặt `-chmod -R 744`, chạy lại jar:

```
hdfs dfs -chmod -R 744 /hcmus/23120166
java -jar hadoop-test.jar 9000 /hcmus/23120166
```

Listing 15: Chạy thử jar lần 2 sau khi cấp quyền

Nguyên nhân: Lỗi `The owner ... is not khtn_23120166` — cần đổi chủ sở hữu đệ quy bằng `-chown -R`.

```
aurelthane@Admin:~$ hdfs dfs -chmod -R 744 /hcmus/23120166
aurelthane@Admin:~$ java -jar hadoop-test.jar 9000 /hcmus/23120166
Trying to read /hcmus/23120166
log4j:WARN No appenders could be found for logger (org.apache.hadoop.util.Shell).
log4j:WARN Please initialize the log4j system properly.
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.html#noconfig for more info.
Found hdfs://localhost:9000/hcmus/23120166/test_file.txt
The owner of hdfs://localhost:9000/hcmus/23120166/test_file.txt is not khtn_23120166
```

Hình 24: Lần chạy 2: Quyền 744 đã đúng nhưng chủ sở hữu file HDFS vẫn chưa khớp với `khtn_23120166`

2.10.4 Lần chạy 3 — Thành công, sinh file Verification

```
hdfs dfs -chown -R khtn_23120166 /hcmus/23120166
java -jar hadoop-test.jar 9000 /hcmus/23120166
```

Listing 16: Sửa quyền chủ sở hữu đệ quy và chạy jar lần 3

Lưu ý về cảnh báo mạng: Trong output của lần chạy thành công, xuất hiện hai dòng cảnh báo:

> *The first method to get MAC address is failed: Could not get network interface*
>
> *Trying the alternative method*

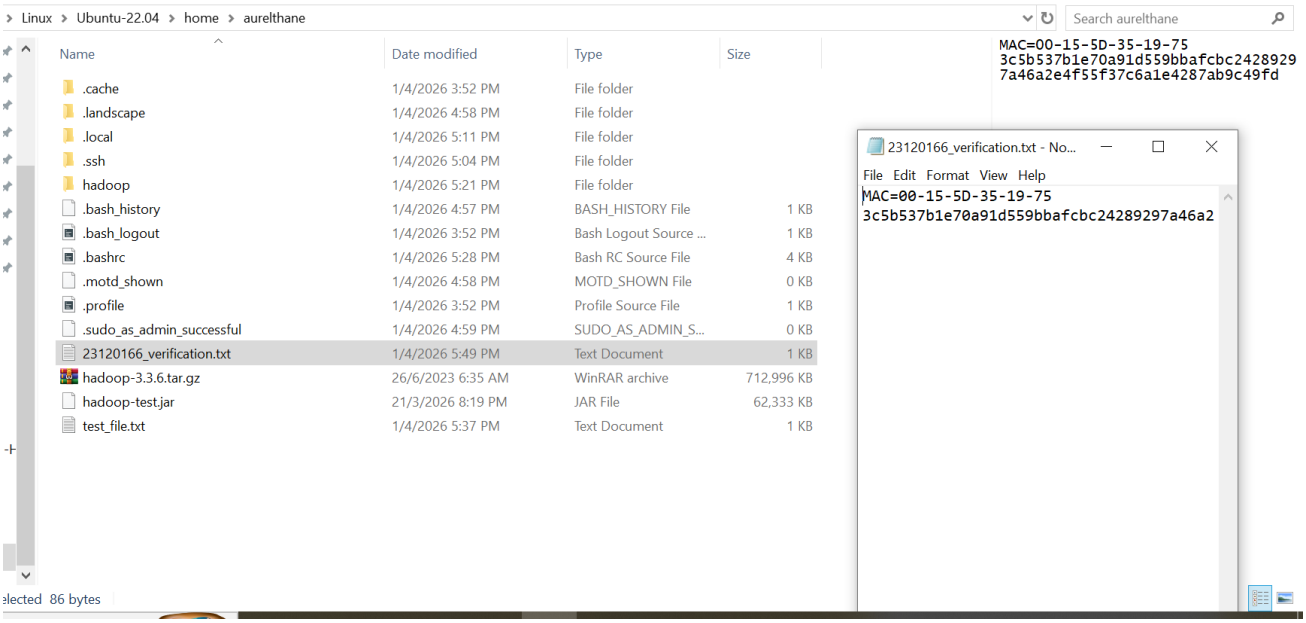
Đây là hành vi bình thường trong môi trường WSL. Do kiến trúc mạng ảo hóa của WSL, máy ảo Java không thể truy xuất địa chỉ MAC trực tiếp qua giao diện mạng thông thường. Tuy nhiên, chương trình `hadoop-test.jar` đã được lập trình sẵn cơ chế dự phòng (alternative method) để thu thập thông tin mạng theo cách khác, và vẫn xuất ra file xác thực thành công.

```
aurelthane@Admin:~$ hdfs dfs -chmod -R 744 /hcmus/23120166
aurelthane@Admin:~$ java -jar hadoop-test.jar 9000 /hcmus/23120166
Trying to read /hcmus/23120166
log4j:WARN No appenders could be found for logger (org.apache.hadoop.util.Shell).
log4j:WARN Please initialize the log4j system properly.
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.html#noconfig for more info.
Found hdfs://localhost:9000/hcmus/23120166/test_file.txt
The owner of hdfs://localhost:9000/hcmus/23120166/test_file.txt is not khtn_23120166
aurelthane@Admin:~$ ls
hadoop  hadoop-3.3.6.tar.gz  hadoop-test.jar  test_file.txt
aurelthane@Admin:~$ pwd
/home/aurelthane
aurelthane@Admin:~$ hdfs dfs -chown -R khtn_23120166 /hcmus/23120166
aurelthane@Admin:~$ java -jar hadoop-test.jar 9000 /hcmus/23120166
Trying to read /hcmus/23120166
log4j:WARN No appenders could be found for logger (org.apache.hadoop.util.Shell).
log4j:WARN Please initialize the log4j system properly.
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.html#noconfig for more info.
Found hdfs://localhost:9000/hcmus/23120166/test_file.txt
Your student ID: 23120166 (ensure it matches your student ID)
The first method to get MAC address is failed: Could not get network interface
Trying the alternative method
The first method to get MAC address is failed: Could not get network interface
Trying the alternative method
File written at /home/aurelthane/23120166_verification.txt
aurelthane@Admin:~$ ls
23120166_verification.txt  hadoop  hadoop-3.3.6.tar.gz  hadoop-test.jar  test_file.txt
```

Hình 25: Lần chạy 3: Chương trình trả về *"Your student ID: 23120166"* — cảnh báo MAC bình thường trong WSL, cơ chế dự phòng hoạt động và sinh file `23120166_verification.txt` thành công

2.10.5 Kiểm tra file kết quả

File `23120166_verification.txt` chứa địa chỉ MAC của máy ảo WSL, dùng làm bằng chứng thực hành.



Hình 26: File `23120166_verification.txt` chứa địa chỉ MAC `00-15-5D-35-19-75` — minh chứng hoàn thành bài thực hành trên máy cụ thể

3 Bài toán đếm từ (Word Count) với bộ lọc ký tự đầu bằng Scala

Phần này trình bày chi tiết quá trình xây dựng, biên dịch và chạy một chương trình MapReduce bằng Scala trên nền tảng Hadoop. Yêu cầu bài toán không phải đếm tất cả các từ, mà **chỉ đếm những từ có ký tự đầu tiên thuộc tập hợp cố định**: `{f, i, t, h, c, m, u, s}` — lấy cảm hứng từ tên viết tắt của Khoa Công nghệ Thông tin, Đại học Khoa học Tự nhiên (FIT HCMUS).

3.1 Bước 1: Chuẩn bị dữ liệu đầu vào trên HDFS

Mục đích: Trong quy trình MapReduce, dữ liệu đầu vào phải nằm trên HDFS để Hadoop có thể phân tách thành các InputSplit và phân phối đến các Mapper node. File `words.txt` (5.3 MB) là bộ văn bản thô dùng làm dữ liệu đầu vào.

Các lệnh thực hiện:

1. `hdfs dfs -mkdir`: Tạo thư mục `/hcmus/23120166/input` trên HDFS để chứa file đầu vào.
2. `hdfs dfs -put`: Đẩy file `words.txt` từ filesystem local lên HDFS.
3. `hdfs dfs -ls`: Kiểm tra file đã được upload thành công với đúng kích thước.

```
hdfs dfs -mkdir /hcmus/23120166/input
hdfs dfs -put words.txt /hcmus/23120166/input/
hdfs dfs -ls /hcmus/23120166/input/
```

Listing 17: Tạo thư mục input và upload words.txt lên HDFS

```
aurelthane@Admin:~$ hdfs dfs -mkdir /hcmus/23120166/input
aurelthane@Admin:~$ ls
23120166_verification.txt  hadoop  hadoop-3.3.6.tar.gz  hadoop-test.jar  test_file.txt  words.txt
aurelthane@Admin:~$ hdfs dfs -put words.txt /hcmus/23120166/input/
aurelthane@Admin:~$ hdfs dfs -ls /hcmus/23120166/input/
Found 1 items
-rw-r--r--   1 aurelthane supergroup    5329535 2026-04-01 19:12 /hcmus/23120166/input/words.txt
aurelthane@Admin:~$
```

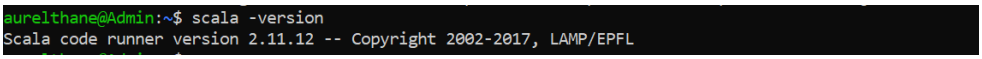
Hình 27: File `words.txt` (5,329,535 bytes) đã được upload thành công vào `/hcmus/23120166/input/` trên HDFS

3.2 Bước 2: Kiểm tra phiên bản Scala

Mục đích: Xác nhận Scala đã được cài đặt và phiên bản phù hợp với Hadoop. Scala 2.11.x là phiên bản ổn định tương thích tốt với Hadoop 3.x trên Java 11.

```
scala -version
```

Listing 18: Kiểm tra phiên bản Scala



Hình 28: Scala 2.11.12 đã được cài đặt sẵn trên hệ thống — phiên bản tương thích với môi trường Hadoop

3.3 Bước 3: Thiết kế chương trình MapReduce bằng Scala

Chương trình được xây dựng thành 3 thành phần chính trong file `FilterWordCount.scala`:

3.3.1 Lớp TokenizerMapper

Nhiệm vụ: Đọc từng dòng văn bản, tách token theo khoảng trắng, lọc những từ có ký tự đầu thuộc tập mục tiêu, và phát ra cặp `(ký_tự, 1)` cho từng từ thỏa điều kiện.

```scala
class TokenizerMapper extends Mapper[Object, Text, Text, IntWritable] {
    private val one = new IntWritable(1)
    private val outKey = new Text()
    private val targetChars = Set('f', 'i', 't', 'h', 'c', 'm', 'u', 's')

    override def map(key: Object, value: Text,
            context: Mapper[Object, Text, Text, IntWritable]#Context
    ): Unit = {
        val tokenizer = new StringTokenizer(value.toString)
        while (tokenizer.hasMoreTokens) {
            val token = tokenizer.nextToken().toLowerCase
            if (token.nonEmpty) {
                val firstLetterOpt = token.find(_.isLetter)
                firstLetterOpt match {
                    case Some(ch) if targetChars.contains(ch) =>
                        outKey.set(ch.toString)
                        context.write(outKey, one)
                    case _ =>
                }
            }
        }
    }
```

```
}
```

Listing 19: Lớp TokenizerMapper — lọc và phát cặp key-value

3.3.2 Lớp IntSumReducer

Nhiệm vụ: Tổng hợp số lần xuất hiện của mỗi ký tự đầu. Sử dụng `LinkedHashMap` để duy trì thứ tự xuất ra theo đúng thứ tự `f, i, t, h, c, m, u, s` thay vì thứ tự bảng chữ cái mặc định của Hadoop shuffle.

```
class IntSumReducer extends Reducer[Text, IntWritable, Text, IntWritable] {
    private val result = new IntWritable()
    private val targetChars = Vector('f', 'i', 't', 'h', 'c', 'm', 'u', 's')
    private val counts = mutable.LinkedHashMap(
        targetChars.map(ch => ch -> 0): _*
    )

    override def reduce(key: Text, values: java.lang.Iterable[IntWritable],
            context: Reducer[Text, IntWritable, Text, IntWritable]#Context
    ): Unit = {
        var sum = 0
        val iterator = values.iterator()
        while (iterator.hasNext) { sum += iterator.next().get() }
        val first = key.toString.charAt(0)
        if (counts.contains(first)) { counts.update(first, sum) }
    }

    override def cleanup(
            context: Reducer[Text, IntWritable, Text, IntWritable]#Context
    ): Unit = {
        counts.foreach { case (ch, sum) =>
            result.set(sum)
            context.write(new Text(ch.toString), result)
        }
    }
}
```

Listing 20: Lớp IntSumReducer — gom cụm và duy trì thứ tự xuất ra

3.3.3 Object FilterWordCount (Driver)

Nhiệm vụ: Cấu hình và submit Job lên YARN. Chỉ định đường dẫn input/output, gán các lớp Mapper, Reducer và kiểu dữ liệu đầu ra.

```scala
object FilterWordCount {
    def main(args: Array[String]): Unit = {
        val conf = new Configuration()
        val job = Job.getInstance(conf, "filter word count fit hcmus")
        job.setJarByClass(this.getClass)
        job.setMapperClass(classOf[TokenizerMapper])
        job.setReducerClass(classOf[IntSumReducer])
        job.setNumReduceTasks(1)
        job.setOutputKeyClass(classOf[Text])
        job.setOutputValueClass(classOf[IntWritable])
        FileInputFormat.addInputPath(job, new Path(args(0)))
        FileOutputFormat.setOutputPath(job, new Path(args(1)))
        System.exit(if (job.waitForCompletion(true)) 0 else 1)
    }
}
```

Listing 21: Driver — cấu hình và submit MapReduce Job lên YARN

3.4 Bước 4: Biên dịch mã nguồn Scala và đóng gói JAR

Mục đích: Chuyển mã nguồn Scala thành bytecode JVM (`.class`), sau đó đóng gói tất cả thành một file `.jar` di động để Hadoop có thể phân phối đến các node worker.

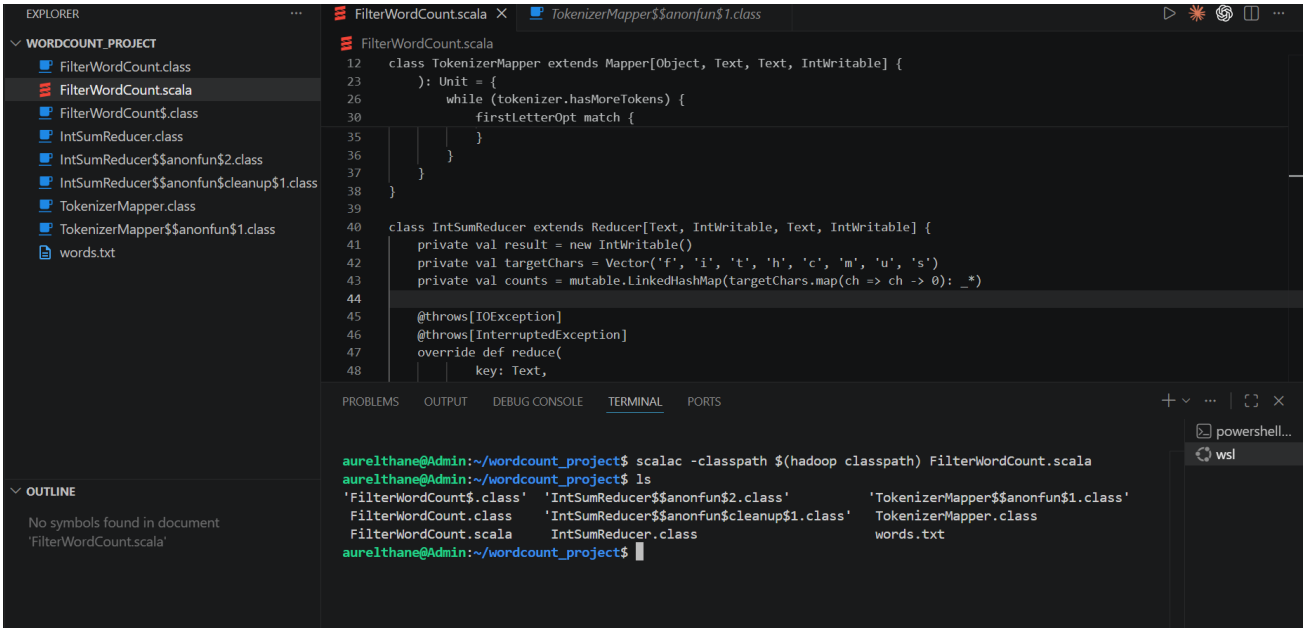
Các lệnh thực hiện:

1. `scalac -classpath $(hadoop classpath)`: Biên dịch Scala với classpath đầy đủ của Hadoop để trình biên dịch nhận biết các thư viện `org.apache.hadoop.*`.

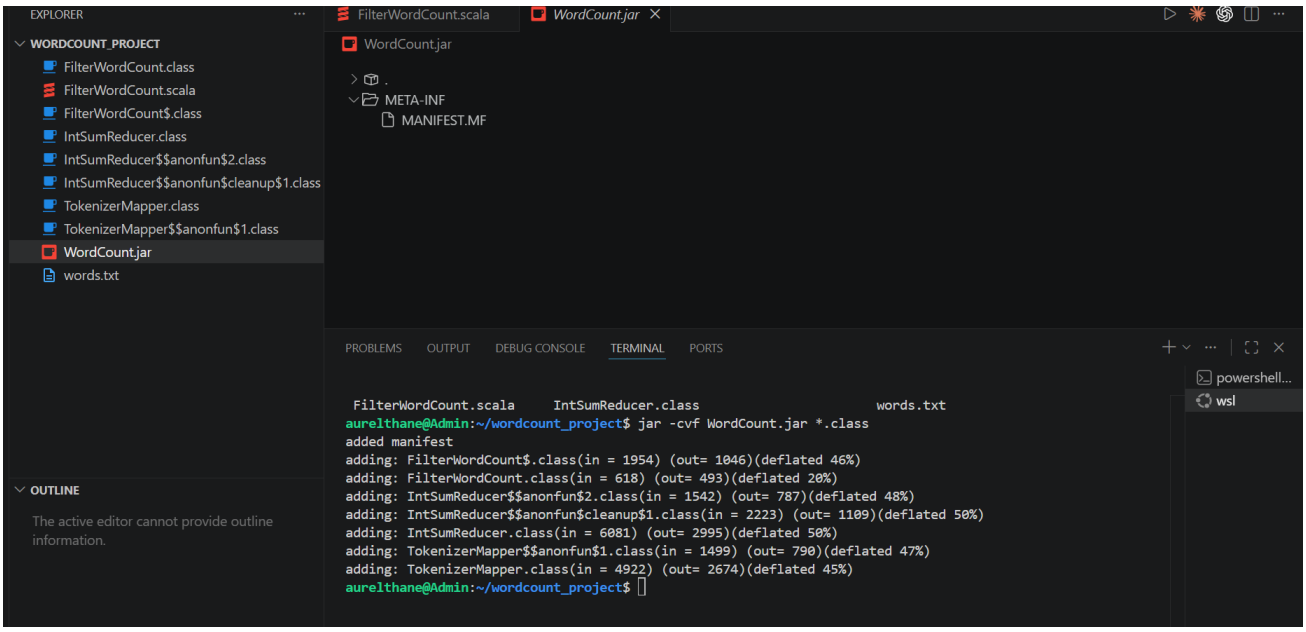
2. `jar -cvf`: Đóng gói tất cả file `*.class` thành `WordCount.jar`.

```bash
mkdir wordcount_project && cd wordcount_project
scalac -classpath $(hadoop classpath) FilterWordCount.scala
ls
jar -cvf WordCount.jar *.class
```

Listing 22: Biên dịch Scala và đóng gói thành WordCount.jar



Hình 29: VSCode hiển thị kết quả biên dịch — 7 file `.class` được tạo ra tương ứng với các lớp và anonymous functions



Hình 30: Lệnh `jar -cvf` đóng gói thành công tất cả các file `.class` vào `WordCount.jar`

3.5 Bước 5: Chạy MapReduce Job và xử lý sự cố ClassPath

3.5.1 Lần chạy 1 — Lỗi ClassNotFoundException (Scala Runtime)

```
hadoop jar WordCount.jar FilterWordCount /hcmus/23120166/input /hcmus
    /23120166/output
```

Listing 23: Chạy MapReduce Job lần đầu

Vấn đề: Job thất bại với lỗi `java.lang.NoClassDefFoundError: scala/collection/Seq`.

Nguyên nhân: Thư viện runtime của Scala (`scala-library.jar`) không nằm trong classpath của Hadoop khi khởi động Job. Hadoop chỉ load các thư viện trong `$HADOOP_HOME/lib/` và classpath mặc định, không bao gồm thư viện Scala đã cài trên hệ thống.

```
2026-04-01 20:46:09,511 INFO output.FileOutputCommitter: File Output Committer Algorithm version is 2
2026-04-01 20:46:09,511 INFO output.FileOutputCommitter: FileOutputCommitter skip cleanup _temporary folders under output directory:false, ignore
2026-04-01 20:46:09,512 INFO mapred.LocalJobRunner: OutputCommitter is org.apache.hadoop.mapreduce.lib.output.FileOutputCommitter
2026-04-01 20:46:09,554 INFO mapred.LocalJobRunner: Waiting for map tasks
2026-04-01 20:46:09,554 INFO mapred.LocalJobRunner: Starting task: attempt_local145315622_0001_m_000000_0
2026-04-01 20:46:09,572 INFO output.PathOutputCommitterFactory: No output committer factory defined, defaulting to FileOutputCommitterFactory
2026-04-01 20:46:09,572 INFO output.FileOutputCommitter: File Output Committer Algorithm version is 2
2026-04-01 20:46:09,573 INFO output.FileOutputCommitter: FileOutputCommitter skip cleanup _temporary folders under output directory:false, ignore
2026-04-01 20:46:09,601 INFO mapred.Task:  Using ResourceCalculatorProcessTree : [ ]
2026-04-01 20:46:09,602 INFO mapred.LocalJobRunner: map task executor complete.
2026-04-01 20:46:09,636 WARN mapred.LocalJobRunner: job_local145315622_0001
java.lang.Exception: java.lang.NoClassDefFoundError: scala/collection/Seq
        at org.apache.hadoop.mapred.LocalJobRunner$Job.runTasks(LocalJobRunner.java:492)
        at org.apache.hadoop.mapred.LocalJobRunner$Job.run(LocalJobRunner.java:552)
Caused by: java.lang.NoClassDefFoundError: scala/collection/Seq
        at java.base/java.lang.Class.forName0(Native Method)
        at java.base/java.lang.Class.forName(Class.java:398)
        at org.apache.hadoop.conf.Configuration.getClassByNameOrNull(Configuration.java:2630)
        at org.apache.hadoop.conf.Configuration.getClassByName(Configuration.java:2595)
        at org.apache.hadoop.conf.Configuration.getClass(Configuration.java:2691)
        at org.apache.hadoop.mapreduce.task.JobContextImpl.getMapperClass(JobContextImpl.java:189)
        at org.apache.hadoop.mapred.MapTask.runNewMapper(MapTask.java:760)
        at org.apache.hadoop.mapred.MapTask.run(MapTask.java:348)
        at org.apache.hadoop.mapred.LocalJobRunner$Job$MapTaskRunnable.run(LocalJobRunner.java:271)
        at java.base/java.util.concurrent.Executors$RunnableAdapter.call(Executors.java:515)
        at java.base/java.util.concurrent.FutureTask.run(FutureTask.java:264)
        at java.base/java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1128)
        at java.base/java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:628)
        at java.base/java.lang.Thread.run(Thread.java:829)
Caused by: java.lang.ClassNotFoundException: scala.collection.Seq
        at java.base/java.net.URLClassLoader.findClass(URLClassLoader.java:476)
        at java.base/java.lang.ClassLoader.loadClass(ClassLoader.java:594)
        at java.base/java.lang.ClassLoader.loadClass(ClassLoader.java:527)
        ... 14 more
2026-04-01 20:46:10,503 INFO mapreduce.Job: Job job_local145315622_0001 running in uber mode : false
2026-04-01 20:46:10,506 INFO mapreduce.Job:  map 0% reduce 0%
2026-04-01 20:46:10,510 INFO mapreduce.Job: Job job_local145315622_0001 failed with state FAILED due to: NA
2026-04-01 20:46:10,515 INFO mapreduce.Job: Counters: 0
aurelthane@Admin:~/wordcount_project$
```

Hình 31: MapReduce Job thất bại với `NoClassDefFoundError: scala/collection/Seq` — thiếu Scala Runtime Library trong Hadoop classpath

3.5.2 Tra cứu giải pháp

Tham khảo Stack Overflow, giải pháp được đề xuất là export biến `HADOOP_CLASSPATH` trỏ tới file `scala-library.jar` trước khi chạy lệnh `hadoop jar`.

Hình 32: Giải pháp từ Stack Overflow: thêm `scala-library.jar` vào `HADOOP_CLASSPATH` để Hadoop tìm thấy Scala Runtime

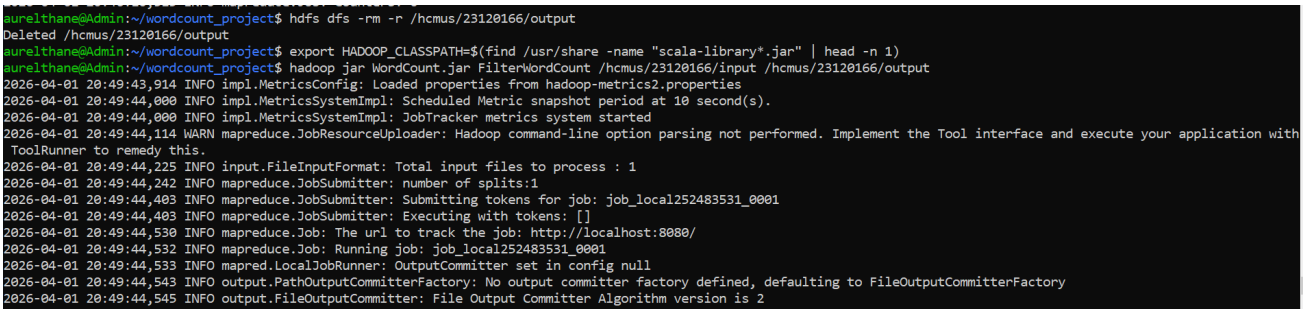
3.5.3 Phương án khắc phục

Tìm vị trí file `scala-library.jar` trong hệ thống và xuất vào `HADOOP_CLASSPATH`:

```
hdfs dfs -rm -r /hcmus/23120166/output
export HADOOP_CLASSPATH=$(find /usr/share -name "scala-library*.jar" | head -n 1)
hadoop jar WordCount.jar FilterWordCount /hcmus/23120166/input /hcmus/23120166/output
```

Listing 24: Export HADOOP_CLASSPATH với Scala Runtime và chạy lại Job

Lưu ý: Thư mục output cũ cần xóa trước khi chạy lại vì Hadoop không cho phép ghi đè vào thư mục đã tồn tại.

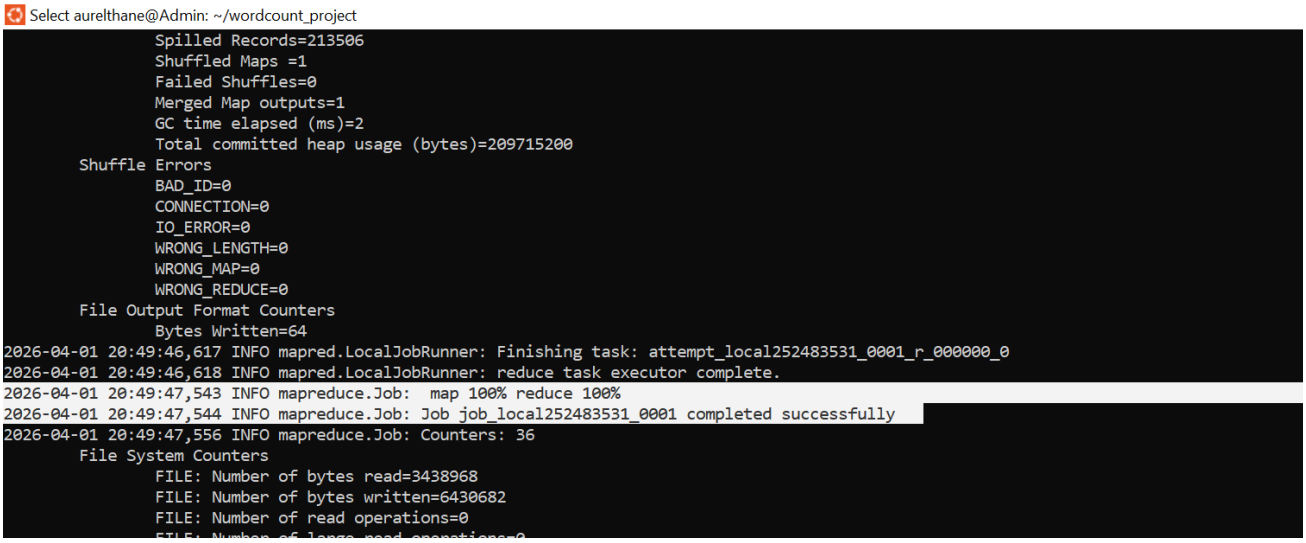


```
aurelthane@Admin:~/wordcount_project$ hdfs dfs -rm -r /hcmus/23120166/output
Deleted /hcmus/23120166/output
aurelthane@Admin:~/wordcount_project$ export HADOOP_CLASSPATH=$(find /usr/share -name "scala-library*.jar" | head -n 1)
aurelthane@Admin:~/wordcount_project$ hadoop jar WordCount.jar FilterWordCount /hcmus/23120166/input /hcmus/23120166/output
2026-04-01 20:49:43,914 INFO impl.MetricsConfig: Loaded properties from hadoop-metrics2.properties
2026-04-01 20:49:44,000 INFO impl.MetricsSystemImpl: Scheduled Metric snapshot period at 10 second(s).
2026-04-01 20:49:44,000 INFO impl.MetricsSystemImpl: JobTracker metrics system started
2026-04-01 20:49:44,114 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with
 ToolRunner to remedy this.
2026-04-01 20:49:44,225 INFO input.FileInputFormat: Total input files to process : 1
2026-04-01 20:49:44,242 INFO mapreduce.JobSubmitter: number of splits:1
2026-04-01 20:49:44,403 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_local252483531_0001
2026-04-01 20:49:44,403 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-04-01 20:49:44,530 INFO mapreduce.Job: The url to track the job: http://localhost:8080/
2026-04-01 20:49:44,532 INFO mapreduce.Job: Running job: job_local252483531_0001
2026-04-01 20:49:44,533 INFO mapred.LocalJobRunner: OutputCommitter set in config null
2026-04-01 20:49:44,543 INFO output.PathOutputCommitterFactory: No output committer factory defined, defaulting to FileOutputCommitterFactory
2026-04-01 20:49:44,545 INFO output.FileOutputCommitter: File Output Committer Algorithm version is 2
```

Hình 33: Sau khi export `HADOOP_CLASSPATH` với `scala-library.jar`, Job được submit thành công và bắt đầu thực thi

3.6 Bước 6: Theo dõi tiến trình và kết quả Job

Mục đích: Xác nhận Job hoàn thành thành công, theo dõi tỉ lệ hoàn thành của pha Map và Reduce, kiểm tra các counter thống kê.



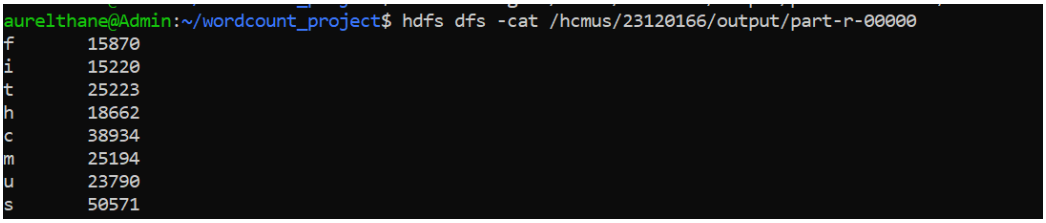
Hình 34: Job hoàn thành với `map 100% reduce 100%` và thông báo *"Job completed successfully"* — tất cả 36 counters được ghi nhận

3.7 Bước 7: Đọc và lưu kết quả

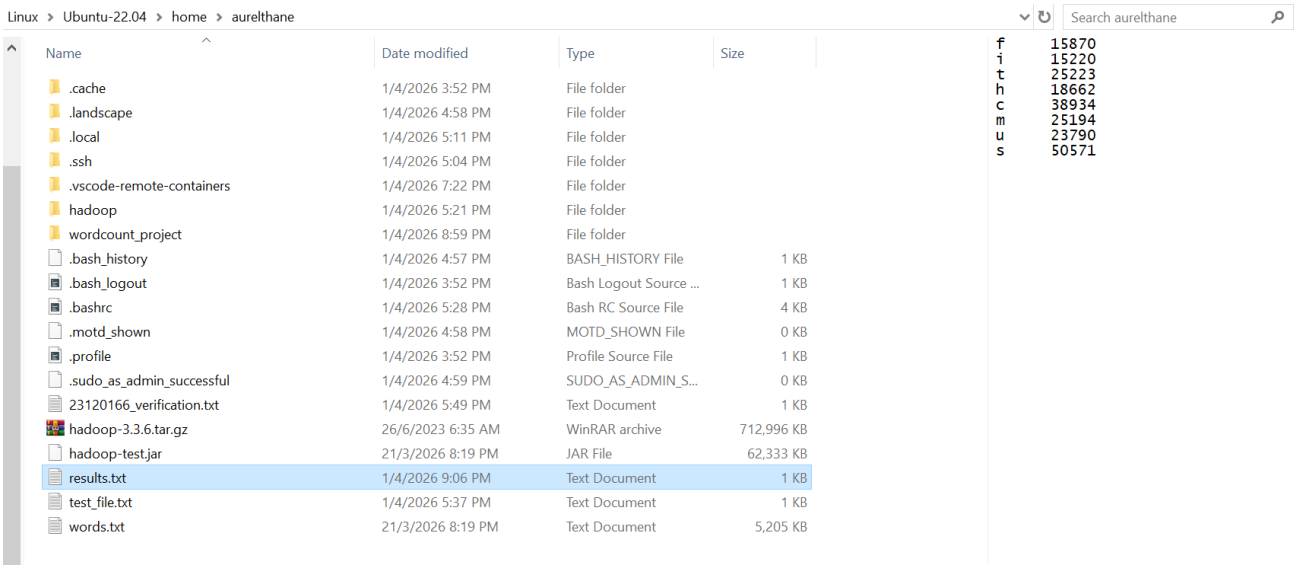
Mục đích: Đọc kết quả từ HDFS và lưu về máy local để kiểm tra.

```
hdfs dfs -cat /hcmus/23120166/output/part-r-00000
```

Listing 25: Đọc kết quả từ HDFS output



Hình 35: Kết quả đếm từ: `f=15870`, `i=15220`, `t=25223`, `h=18662`, `c=38934`, `m=25194`, `u=23790`, `s=50571` — xuất theo đúng thứ tự FIT HCMUS



Hình 36: Windows Explorer xác nhận file `results.txt` đã được lưu về máy local, cùng với kết quả đếm từ hiển thị bên phải

Nhận xét kết quả: Các từ bắt đầu bằng ký tự `s` xuất hiện nhiều nhất (50,571 lần), tiếp theo là `c` (38,934 lần). Thứ tự xuất ra đúng theo chuỗi `f, i, t, h, c, m, u, s` nhờ thiết kế dùng `LinkedHashMap` trong `IntSumReducer.cleanup()`, cho thấy cơ chế ghi đè thứ tự shuffle của Hadoop hoạt động đúng như thiết kế.

4 Kết luận

Qua quá trình thực hành Lab 1, đã hoàn thành được các mục tiêu đề ra:

4.1 Tóm tắt kết quả

1. **Cài đặt Hadoop Pseudo-Distributed Mode thành công:**
   - Cài đặt và cấu hình Java 11, Hadoop 3.3.6 trên WSL/Ubuntu 22.04.
   - Cấu hình SSH không cần mật khẩu cho giao tiếp nội bộ.
   - Cấu hình đầy đủ 4 file XML: `core-site.xml`, `hdfs-site.xml`, `mapred-site.xml`, `yarn-site.xml`.
   - Khởi động thành công 5 daemon: NameNode, DataNode, SecondaryNameNode, ResourceManager, NodeManager.
   - Xác nhận hoạt động qua Web UI (`localhost:9870`) và lệnh `jps`.
2. **Thực hiện bài toán Word Count với Scala MapReduce thành công:**

- Biên dịch file Scala với Hadoop classpath bằng `scalac`.
- Đóng gói thành `WordCount.jar`.
- Chạy MapReduce job trên HDFS và nhận kết quả đếm từ theo ký tự đầu.

3. **Xử lý các lỗi phát sinh trong quá trình thực hành:**
   - Lỗi `pdsh: rcmd: Permission denied` → khắc phục bằng `PDSH_RCMD_TYPE=ssh`.
   - Lỗi `NoClassDefFoundError: scala/collection/Seq` → khắc phục bằng `HADOOP_CLASSPATH`.

4.2 Kinh nghiệm rút ra

- **Về cấu hình môi trường:** Việc thiết lập đúng biến môi trường (`JAVA_HOME`, `HADOOP_HOME`, `PDSH_RCMD_TYPE`) là nền tảng quan trọng để Hadoop hoạt động ổn định.
- **Về phát triển MapReduce với Scala:** Khi sử dụng ngôn ngữ JVM như Scala với Hadoop, cần chú ý đến việc cung cấp đủ thư viện runtime khi chạy job. Sử dụng `HADOOP_CLASSPATH` là cách đơn giản và hiệu quả để giải quyết vấn đề phụ thuộc thư viện.
- **Về thiết kế MapReduce:** Việc dùng `cleanup()` trong Reducer và `LinkedHashMap` để đảm bảo thứ tự đầu ra là một kỹ thuật quan trọng khi cần kết quả có thứ tự xác định.
- **Về debugging:** Stack Overflow và tài liệu chính thức của Hadoop là nguồn tài nguyên quan trọng khi gặp lỗi kỹ thuật.

A Phụ lục – Mã nguồn đầy đủ

A.1 FilterWordCount.scala

```scala
import java.io.IOException
import java.util.StringTokenizer
import scala.collection.mutable

import org.apache.hadoop.conf.Configuration
import org.apache.hadoop.fs.Path
import org.apache.hadoop.io.{IntWritable, Text}
import org.apache.hadoop.mapreduce.{Job, Mapper, Reducer}
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat

class TokenizerMapper extends Mapper[Object, Text, Text, IntWritable] {
    private val one = new IntWritable(1)
    private val outKey = new Text()
    private val targetChars = Set('f', 'i', 't', 'h', 'c', 'm', 'u', 's')

    @throws[IOException]
    @throws[InterruptedException]
    override def map(
            key: Object,
            value: Text,
            context: Mapper[Object, Text, Text, IntWritable]#Context
    ): Unit = {
        val tokenizer = new StringTokenizer(value.toString)
        while (tokenizer.hasMoreTokens) {
            val token = tokenizer.nextToken().toLowerCase
            if (token.nonEmpty) {
                val firstChar = token.charAt(0)
                if (targetChars.contains(firstChar)) {
                    outKey.set(firstChar.toString)
                    context.write(outKey, one)
                }
            }
        }
    }
}
```

```

class IntSumReducer extends Reducer[Text, IntWritable, Text, IntWritable] {
    private val result = new IntWritable()
    private val targetChars = Vector('f', 'i', 't', 'h', 'c', 'm', 'u', 's')
    private val counts = mutable.LinkedHashMap(
        targetChars.map(ch => ch -> 0): _*
    )

    @throws[IOException]
    @throws[InterruptedException]
    override def reduce(
            key: Text,
            values: java.lang.Iterable[IntWritable],
            context: Reducer[Text, IntWritable, Text, IntWritable]#Context
    ): Unit = {
        var sum = 0
        val iterator = values.iterator()
        while (iterator.hasNext) {
            sum += iterator.next().get()
        }
        val first = key.toString.charAt(0)
        if (counts.contains(first)) {
            counts.update(first, sum)
        }
    }

    @throws[IOException]
    @throws[InterruptedException]
    override def cleanup(
            context: Reducer[Text, IntWritable, Text, IntWritable]#Context
    ): Unit = {
        counts.foreach { case (ch, sum) =>
            result.set(sum)
            context.write(new Text(ch.toString), result)
        }
    }
}

object FilterWordCount {
    def main(args: Array[String]): Unit = {
```

```
        if (args.length != 2) {
            System.err.println("Usage: FilterWordCount <input_path> <output_path>")
            System.exit(2)
        }

        val conf = new Configuration()
        val job = Job.getInstance(conf, "filter word count fit hcmus")

        job.setJarByClass(this.getClass)
        job.setMapperClass(classOf[TokenizerMapper])
        job.setReducerClass(classOf[IntSumReducer])
        job.setNumReduceTasks(1)

        job.setOutputKeyClass(classOf[Text])
        job.setOutputValueClass(classOf[IntWritable])

        FileInputFormat.addInputPath(job, new Path(args(0)))
        FileOutputFormat.setOutputPath(job, new Path(args(1)))

        System.exit(if (job.waitForCompletion(true)) 0 else 1)
    }
}
```

Listing 26: Mã nguồn đầy đủ FilterWordCount.scala